

BMIT3173 Integrative Programming

ASSIGNMENT 202605

Student Name : Foo Chong Xian

Student ID : 25WMR09721

Programme : Bachelor in Information Technology (Honours)
(Information Security)

Tutorial Group : Group 4

System Title : LearnSync - Integrated Educational Resource and
Collaborative Learning Portal

Chosen SDG : SDG 4 - Quality Education

Modules : Module 2 – Academic Resources Repository

AI Tools Usage Policy & Disclosure

This assignment is governed by the **TAR UMT Policy for the Use of Artificial Intelligence (AI) (PO/111:26)** and is classified under the **YELLOW (Limited AI)** category. AI tools may be used only for the purposes listed below, not to produce the technical work, analysis, or original content being assessed.

Mandatory submission: Every student must complete and submit the **AI Usage Disclosure Form** (below) with the report, whether or not AI tools were used, by 6th September 2026. A report submitted without the Form is incomplete and may not be marked.

Permitted (Yellow) uses of AI tools:

- Language refinement (grammar, spelling, clarity) of text you wrote, without altering its technical meaning.
- Clarifying taught concepts (e.g. how a design pattern, web-service protocol, or attack works) to aid your understanding.
- Debugging help on code you wrote, where AI only locates or explains an error you then fix yourself.
- Brainstorming early ideas (e.g. SDG framings or topic angles), provided the chosen direction and its justification are your own.

Prohibited uses of AI tools:

- Generating the PHP code, design-pattern implementation, class diagrams, or web-service code being assessed.
- Producing core arguments or analysis, including the design-pattern justification, threat analysis, and secure-coding rationale.
- Uploading confidential, proprietary, or others' personal data to public AI tools (Sections 2.5 and 5 of the AI Policy).

You remain responsible for the accuracy and originality of your work; AI errors do not excuse incorrect content. The lecturer may ask you to explain or demonstrate any part to verify authorship. Undisclosed or prohibited AI use is a breach of academic integrity under Section 4 of the AI Policy. **Complete the Form below; if AI was used, list each use and cite the tool(s) in your References (APA 7th).**

AI Usage Disclosure Form

Declaration (tick one):

- ☐ No AI tools were used in the preparation of this report.
- ☒ AI tools were used as declared in the table below.

AI Tool Used (name & version)	Purpose / How It Was Used	Report Section(s) Affected
Grammarly (2026 Edition)	Used solely for language refinement, spelling verification, and grammatical sentence flow of my own written text without altering technical logic.	Sections 1, 2, 4.1, 4.3, 5.1

If no AI tools were used, tick "No AI tools used". Non-disclosure breaches the AI Policy.

I declare this Form is true and complete and that my AI use complied with the AI Policy and the Yellow conditions above.

Sign: _____ *Foo* _____

Date: ____ 6/9/2026 ____

Table of Contents

AI Tools Usage Policy & Disclosure	2
1. Introduction to the System	4
1.1 System Overview	4
1.2 The Sustainable Development Goal	4
1.3 System Contribution to SDG 4 & Scope	5
2. Module Description	6
2.1 Scope of Module 2: Academic Resources Repository	6
2.2 Functional Breakdown & Class Paths	6
F2.1: The Course Hub Page	6
F2.2: Enrolment by Invitation or Class Code	7
F2.3: Course Materials Under Four Fixed Headings	8
F2.4: Announcements and Their Comment Threads	9
F2.5: The Calendar	10
3. Entity Classes	12
3.1 Entity Class Diagram	12
Figure 3.1: Entity Class Diagram for Module 2 Academic Resources Repository	12
3.2 Entity Class Implementation (Eloquent ORM Mapping)	13
4. Design Pattern	15
4.1 Description of Design Pattern	15
4.2 Implementation of Design Pattern	16
1. The Target Interface for Materials (app/Patterns/Adapter/DisplayableMaterial.php)	16
2. The Adapter for an Uploaded File (app/Patterns/Adapter/FileResourceAdapter.php)	16
3. The Adapter for an External Link (app/Patterns/Adapter/ExternalResourceAdapter.php)	19
4. Choosing the Right Adapter (app/Patterns/Adapter/MaterialAdapterFactory.php)	21
5. The View, Which Never Asks Which Kind It Has (resources/views/courses/show.blade.php)	21
6. The Second Use of the Pattern (app/Patterns/Adapter/AssignmentDeadlineAdapter.php)	22
4.3 Justification of Design Pattern	24
5. Software Security	25
5.1 Potential Threat/Attack	25
5.2 Secure Coding Practice	27
6. Web Services	31
6.1 Web Service Exposure	31
6.2 Web Service Consumption	35
7. References	39
8. Appendices	40
Appendix A: Automated Testing Results	40
Appendix B: GitHub Repository URL	41

1. Introduction to the System

1.1 System Overview

LearnSync is a web based Learning Management System (LMS). Lecturers use it to upload notes, set quizzes, give assignments and mark student work. Students use it to study the notes, sit the quizzes, hand in assignments, watch their progress, and ask questions in a course forum.

The problem LearnSync solves is about proof. In most systems, the certificate a student earns is just a PDF file. Anyone can open that PDF in a word processor and change the marks. If an employer wants to check whether a certificate is real, they must email the college and wait days for a reply. Most employers do not bother.

LearnSync fixes this. Every certificate it creates carries a unique ID, a QR code anyone can scan, and a hidden security code proving the record has not been changed. Anyone can check a certificate in seconds, for free, without an account.

1.2 The Sustainable Development Goal

LearnSync is built around United Nations Sustainable Development Goal 4: Quality Education (SDG 4).

What SDG 4 aims to do:

SDG 4 wants to make sure education is fair, good quality and available to everyone by 2030 (United Nations, 2015). Target 4.3 is about giving everyone fair access to affordable college and university education. Target 4.4 is about increasing the number of people who have real skills for getting a job.

1.3 System Contribution to SDG 4 & Scope

LearnSync supports SDG 4 in three ways:

1. Who benefits: Students who earn certificates, lecturers who teach and mark the courses, administrators who control access, and outside people such as employers who need to check a qualification.
2. Getting materials to students reliably: Module 2 is the part that makes a course usable at all. Notes, tutorials, practical questions, announcements and the class timetable all live here. Study material is only useful if a student can find it, so everything is filed under fixed headings and shown in a suggested working order rather than as one long list.
3. Making qualifications checkable: Every certificate carries a unique ID, a QR code and a security code, so an employer can confirm it instantly and for free.

What the system does not do: LearnSync does not replace official government accreditation. It makes the certificates a college already gives out easy to check.

2. Module Description

2.1 Scope of Module 2: Academic Resources Repository

I am the developer in charge of Module 2, the Academic Resources Repository Module. This module owns everything a course contains and how a student gets into it. I designed and built:

- the course hub page that gathers materials, quizzes, assignments and announcements in one place
- enrolment by lecturer invitation or by class code, and removal by the lecturer
- course materials filed under four fixed headings, where a material is either an uploaded file or a link to an outside website
- announcements with a comment thread attached to each one
- the calendar, showing scheduled classes and meetings alongside assignment deadlines
- the suggested study plan that turns a course into an ordered list of steps

Two of these involve showing two completely different kinds of thing in one list, which is why this module uses the Adapter pattern, and uses it twice.

2.2 Functional Breakdown & Class Paths

F2.1: The Course Hub Page

- Description: One page gathering everything a course holds: materials under four headings, quizzes, assignments, announcements, a link to the discussion forum, and the badges this subject can earn. A lecturer viewing their own course also sees the class code, the student roster and the invitation box. A student sees the suggested study plan instead, because a lecturer is looking at their course rather than studying it.
- Class Paths:
 - Controller: `app/Http/Controllers/CourseController.php` (index, show, create, store, edit, update, destroy)
 - Model: `app/Models/Course.php`
 - View Template: `resources/views/courses/show.blade.php`

LearnSync

Foo Chong Xian
STUDENT

Log out

Dashboard

Courses

Calendar

Announcements

LEARNING

My certificates

Trophy cabinet

All courses

BMC33404 **Project I**

Ting Hie Choon

Plan, design and deliver a substantial software project, documented from proposal through to evaluation.

Discussion forum

SUGGESTED PLAN 2 of 5 assessed steps done

- Attempt "Project I — Progress Test" QUIZ 4 questions · 15 min
- Submit "Coursework 1" ASSIGNMENT Due 26 Jun, 5:00pm
- Submit "Coursework 2" ASSIGNMENT Due 31 Jul, 5:00pm
- Submit "Lab exercise 2" ASSIGNMENT Due 21 Aug, 2:22am
- Submit "Reading response 3" ASSIGNMENT Due 22 Aug, 1:22am

BADGES FOR THIS SUBJECT

Subject Expert — Project I
Pass every quiz in BMC33404 Project I

ANNOUNCEMENTS

This week's materials are up, and the consultation slot is on the calendar. Bring questions about the assignment rather than saving them for the last day.

Ting Hie Choon · 2 weeks ago

View 4 comments

MATERIALS

LECTURE NOTES (0)

Figure 2.1: The Course Hub Page Seen by a Student

All courses

BMIT3173 **Integrative Programming**

Malarvili A/P Nallayan

Integrate systems using web services, data interchange formats and design patterns within an MVC framework.

Discussion forum

Edit

MATERIALS Add material

LECTURE NOTES (2)

- REST APIs explained — video walkthrough YouTube · youtube.com Remove
- Lecture Note Week 1 PDF · 2.8 MB Remove

TUTORIAL QUESTION (1)

- MDN reference: HTTP request methods Link · developer.mozilla.org Remove

BADGES FOR THIS SUBJECT

Subject Expert — Integrative Programm...
Pass every quiz in BMIT3173 Integrative Programming

CLASS CODE

MN3stH

Anyone with this code can join without an invitation. It is not the course code — give it out only to the class.

Issue a new code

Figure 2.2: The Same Course Seen by Its Lecturer, Showing the Class Code and Roster

F2.2: Enrolment by Invitation or Class Code

- Description: There is no course catalogue and no self service enrolment. A course only appears to a student if one of two things has happened. Either the lecturer invited them, which shows up as an Enrol button on their Courses page, or the lecturer gave them the class code, which is six random characters typed into a join page.
- The class code is not the course code. BMIT3173 is public and printed on every timetable, so letting it grant enrolment would let anyone join anything. The class code is a separate secret, and holding it is itself the proof the lecturer meant you to join. Issuing a new code cancels the old one immediately.
- A student can join a course but cannot leave it. Leaving would let somebody walk away from an assessment and take their submissions and grades out of the lecturer's view. Only the lecturer who owns the course can remove a student.
- Class Paths:
 - Controller: `app/Http/Controllers/EnrolmentController.php` (create, join, store, destroy, removeStudent, invite, withdrawInvitation, rotateClassCode)
 - Model: `app/Models/CourseInvitation.php`
 - View Template: `resources/views/courses/join.blade.php`

Join a course

Courses are not browsable. You get into one either by accepting an invitation from the lecturer, or by entering the class code they gave you here.

You are signed in as



Foo Chong Xian
foochongxian@gmail.com

Class code

Ask your lecturer for the class code, then enter it here.

Join

TO JOIN WITH A CLASS CODE

- Use 6 letters or numbers, with no spaces or symbols
- Capitalisation does not matter
- A code stops working if the lecturer issues a new one
- If you were invited instead, the course is waiting on your [Courses](#) page — no code needed

Figure 2.3: The Join by Class Code Page

STUDENTS (33)

	Foo Chong Xian foochongxian@gmail.com	Remove
	Ong Kwong Wei ongkwongwei@hotmail.com	Remove
	Ong Shun Yan ongshunyan@yahoo.com	Remove
	Serena Lim Sze Kee serenalim@gmail.com	Remove

**Wong Siew Lam**
wongsiewlam@outlook.com

Remove

INVITED, NOT YET ACCEPTED

student1 invited 2 weeks ago	Withdraw
student3 invited 2 weeks ago	Withdraw

INVITE A STUDENT

Invite

Figure 2.4: The Lecturer's Roster with the Invite Box and the Remove Control

F2.3: Course Materials Under Four Fixed Headings

- Description: Materials are filed under four headings that never change: Lecture notes, Tutorial question, Practical question and Others. Every heading always appears with a count, even when it holds nothing, so a student can see that no practical questions have been posted yet rather than wondering whether the section exists.
- A material is one of two completely different things. It is either a file uploaded to the server, which has a size and a type, or a link to a website somebody else owns, which has neither. This is where the Adapter pattern earns its place, and Section 4 covers it in full.
- Class Paths:
 - Controller: `app/Http/Controllers/CourseMaterialController.php` (create, store, destroy)
 - Model: `app/Models/CourseMaterial.php`
 - Adapters: `app/Patterns/Adapter/FileResourceAdapter.php`, `app/Patterns/Adapter/ExternalResourceAdapter.php`
 - View Template: `resources/views/materials/create.blade.php`

← Back to Integrative Programming

Add material

Upload a file or link to something external. Both appear in one list for students — the Adapter pattern makes an external link present itself exactly like an uploaded file.

Title

Category

Lecture notes ▾

Source

☒ Upload a file
PDF, slides, worksheet. Up to 20 MB.

☐ Link to an external resource
A YouTube video, an article, online documentation.

File

Choose File

No file chosen

Add material

Cancel

Figure 2.5: Adding a Material, Showing the Upload and External Link Options

MATERIALS		Add material
LECTURE NOTES (2)		
	REST APIs explained — video walkthrough YouTube · youtube.com	Remove
	Lecture Note Week 1 PDF · 2.8 MB	Remove
TUTORIAL QUESTION (1)		
	MDN reference: HTTP request methods Link · developer.mozilla.org	Remove

Figure 2.6: The Materials List, Where an Uploaded PDF and a YouTube Link Appear in the Same List

F2.4: Announcements and Their Comment Threads

- Description: A lecturer posts an announcement to their course, and an administrator can post one to everybody. Each announcement carries its own comment thread, collapsed behind a View comments link so a busy notice board stays readable.
- Students and lecturers both take part, and the lecturer's own replies are tagged as the author so an answer can be told apart from a classmate's guess. Administrators can read and delete any thread but have no comment box, which follows the rule in the specification that keeps them out of forums. They run the class, they are not in it.
- Class Paths:
 - Controller: `app/Http/Controllers/AnnouncementController.php` (index, create, store, destroy, storeComment, destroyComment)
 - Models: `app/Models/Announcement.php`, `app/Models/AnnouncementComment.php`
 - View Templates: `resources/views/announcements/index.blade.php`, `resources/views/partials/announcement-comments.blade.php`

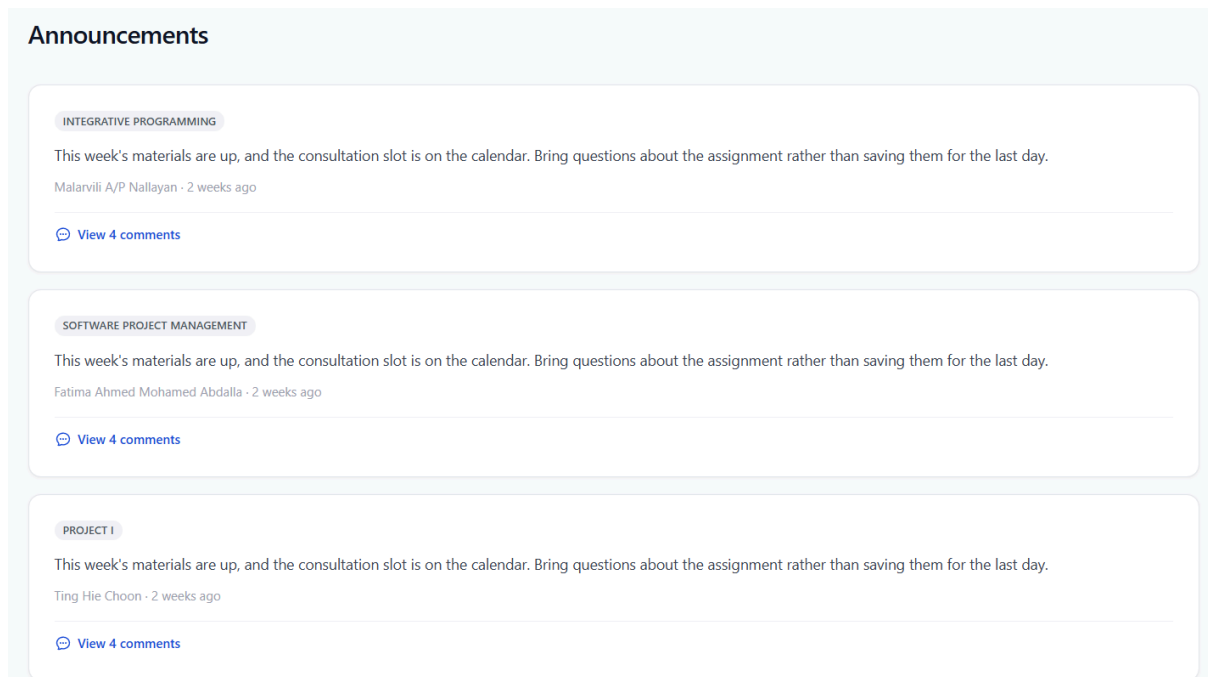


Figure 2.7: The Announcements Page with a Collapsed Comment Thread

Announcements

INTEGRATIVE PROGRAMMING

This week's materials are up, and the consultation slot is on the calendar. Bring questions about the assignment rather than saving them for the last day.

Malarvili A/P Nallayan · 2 weeks ago

View 4 comments

S

Serena Lim Sze Kee · 2 weeks ago

Is the consultation recorded for anyone who cannot make 8pm?

M

Malarvili A/P Nallayan · author · 2 weeks ago

Not recorded, but I will post a summary of anything asked more than once.

F

Foo Chong Xian · 2 weeks ago

Noted, thanks. Is the deadline on the calendar the final one?

Delete

M

Malarvili A/P Nallayan · author · 2 weeks ago

Yes -- what the calendar shows is the assignment's own due date, so it is always current.

F

Add a comment...

Comment

Figure 2.8: An Expanded Thread Showing the Author Tag on the Lecturer's Reply

F2.5: The Calendar

- Description: A month grid showing scheduled classes and online meetings next to assignment deadlines. A lecturer can schedule an event for their own course, and an administrator can schedule one for the whole institution. Students schedule nothing and simply read the calendar for the courses they are enrolled in.
- Assignment deadlines are never copied into the calendar, and that is the important part. They already exist as `assignments.due_date`, which Module 5 owns. A copy would disagree with the original the first time a lecturer moved a deadline. Instead the calendar reads Module 5's assignments and adapts them into the same shape a scheduled event arrives in. Change a due date and the calendar moves with it, because there is only ever one date. This is the Adapter pattern used a second time, on a second pair of mismatched things.
- Clicking any entry opens its own detail page, which shows the times, the room, the description and who it concerns. A meeting shows a Join meeting button. A classroom event has no link, so no button is drawn at all rather than a dead one.
- Class Paths:
 - Controller: `app/Http/Controllers/CalendarController.php` (index, showEvent, createEvent, storeEvent, destroyEvent)
 - Model: `app/Models/CourseEvent.php`
 - Adapters: `app/Patterns/Adapter/ScheduledEventAdapter.php`, `app/Patterns/Adapter/AssignmentDeadlineAdapter.php`
 - View Templates: `resources/views/calendar/index.blade.php`, `resources/views/calendar/show.blade.php`

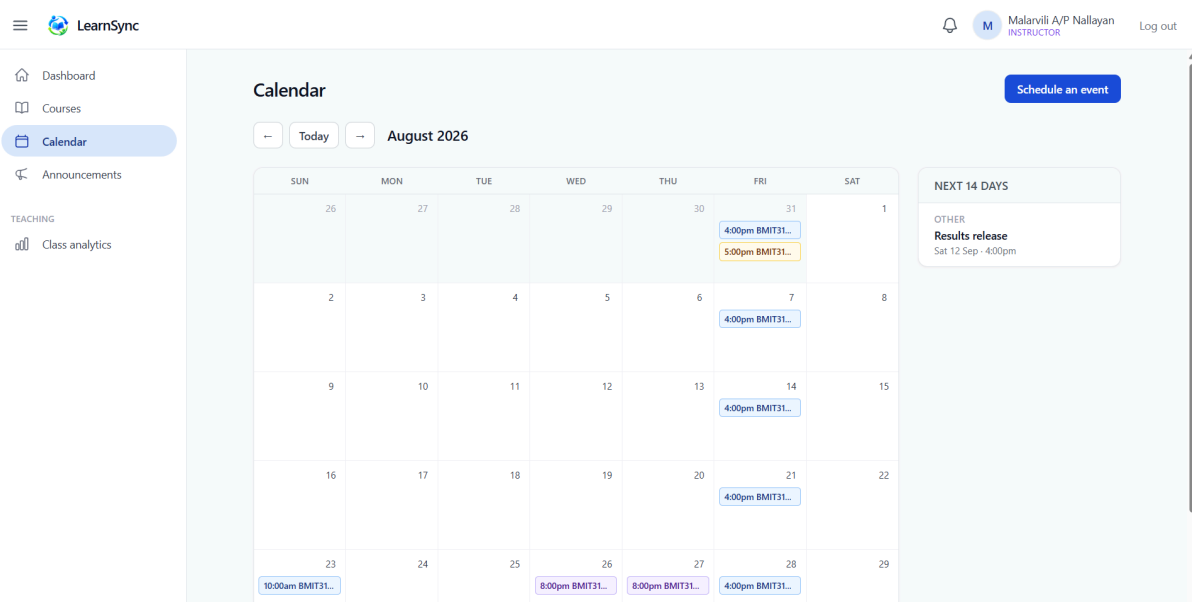


Figure 2.9: The Calendar Month Grid Showing a Class and an Assignment Deadline Together

[← Back to the calendar](#)

ONLINE MEETING

BMIT3173 Integrative Programming

Join meeting 

Consultation (online)

When

Thursday 27 August 2026
8:00pm – 9:00pm (1 hour)
This has already taken place

Organiser

[Malarvili A/P Nallayan](#)

Who it concerns

BMIT3173 Integrative Programming
Foo Chong Xian
Ong Kwong Wei
Ong Shun Yan
Serena Lim Sze Kee

Figure 2.10: An Event Detail Page with the Join Meeting Button

F2.6: The Suggested Study Plan

- **Description:** The course page lists everything a course holds, which answers the question of what is in this course but not the question of what a student should do next. The study plan turns the same content into an order: read the notes, work the tutorial, complete the practical, attempt the quiz, submit the assignment.
- Only the assessed steps report completion. There is no view tracking table, so the system cannot observe whether a student read the notes. Marking a reading step as done would be claiming something the system does not know, so reading steps are always shown as open and the counter at the top counts only what can genuinely be checked.
- **Class Paths:**
 - Support Class: `app/Support/StudyPlan.php` (for, progress, nextIndex)
 - Called from: `app/Http/Controllers/CourseController.php` (show)
 - View Template: `resources/views/partials/study-plan.blade.php`

The screenshot displays a 'SUGGESTED PLAN' section with a progress bar indicating '2 of 5 assessed steps done'. The plan consists of five numbered steps:

Step Number	Task Name	Type	Due Date	Status
1	Attempt "Project 1 — Progress Test"	QUIZ	4 questions · 15 min	Completed (Green checkmark)
2	Submit "Coursework 1"	ASSIGNMENT	Due 26 Jun, 5:00pm	Open (Blue circle with 2)
3	Submit "Coursework 2"	ASSIGNMENT	Due 31 Jul, 5:00pm	Open (Grey circle with 3)
4	Submit "Lab exercise 2"	ASSIGNMENT	Due 21 Aug, 2:22am	Completed (Green checkmark)
5	Submit "Reading response 3"	ASSIGNMENT	Due 22 Aug, 1:22am	Open (Grey circle with 5)

A 'NEXT' button is visible next to the second step.

Figure 2.11: The Suggested Study Plan on a Course Page

3. Entity Classes

3.1 Entity Class Diagram

The diagram below shows the entity classes using object references, meaning one object points to another object, instead of database foreign keys.

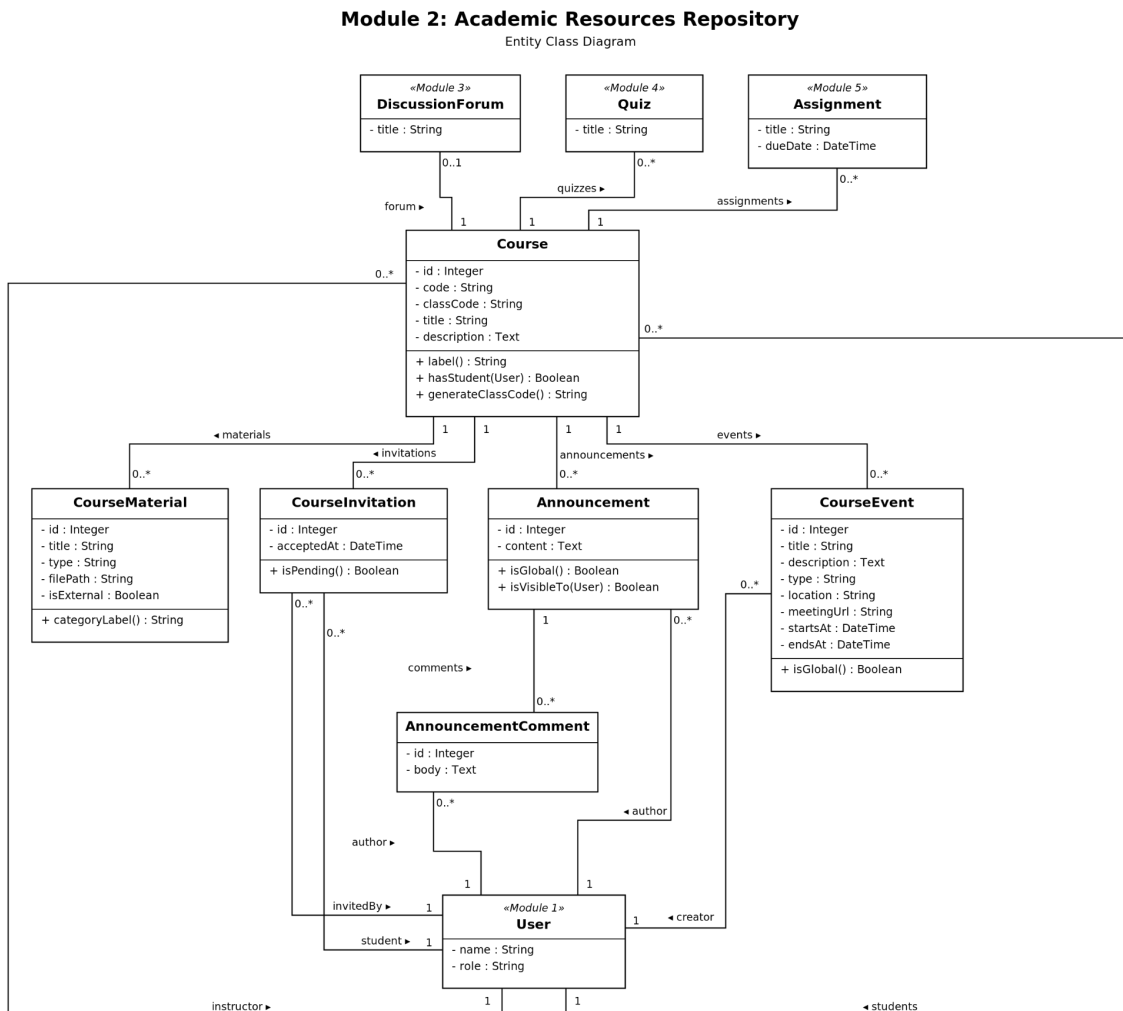


Figure 3.1: Entity Class Diagram for Module 2 Academic Resources Repository

3.2 Entity Class Implementation (Eloquent ORM Mapping)

The classes are written in PHP using Laravel's Eloquent ORM. Relationships are written as methods such as `belongsTo`, `hasMany` and `belongsToMany`, so the code never writes SQL. Calling `$material->course` returns a whole `Course` object, so `$material->course->title` reads the course name by following the link between objects.

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Database\Eloquent\Relations\HasOne;

class Course extends Model
{
    protected $fillable = [
        'instructor_id', 'code', 'class_code', 'title', 'description',
    ];

    // Characters a join code may contain. 0, O, 1, I and l are left out
    // because these codes get read off a slide and typed by hand.
    private const CODE_ALPHABET =
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ23456789';

    // Every course gets a join code the moment it is created, so no
    // controller, seeder or factory has to remember to supply one.
    protected static function booted(): void
    {
        static::creating(function (Course $course) {
            $course->class_code ??= static::generateClassCode();
        });
    }

    /** Object reference: a course is taught by one lecturer */
    public function instructor(): BelongsTo
    {
        return $this->belongsTo(User::class, 'instructor_id');
    }

    /** Object reference: a course has many enrolled students */
    public function students(): BelongsToMany
    {
        return $this->belongsToMany(User::class, 'course_student', 'course_id', 'student_id')
            ->withTimestamps();
    }

    /** Object reference: a course holds many materials */
    public function materials(): HasMany
    {
        return $this->hasMany(CourseMaterial::class);
    }
}
```

```

    }

    /** Object reference: a course has exactly one discussion forum */
    public function forum(): HasOne
    {
        return $this->hasOne(DiscussionForum::class);
    }

    /** The class answers questions about itself, instead of a controller */
    public function hasStudent(User $user): bool
    {
        return $this->students()->whereKey($user->id)->exists();
    }
}

```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class CourseMaterial extends Model
{
    protected $fillable = ['course_id', 'title', 'type', 'file_path', 'is_external'];

    protected function casts(): array
    {
        return ['is_external' => 'boolean'];
    }

    // The four fixed headings the course page always shows.
    public const CATEGORIES = [
        'lecture' => 'Lecture notes',
        'tutorial' => 'Tutorial question',
        'practical' => 'Practical question',
        'other' => 'Others',
    ];

    /** Object reference: a material belongs to one course */
    public function course(): BelongsTo
    {
        return $this->belongsTo(Course::class);
    }

    public function categoryLabel(): string
    {
        return self::CATEGORIES[$this->type] ?? self::CATEGORIES['other'];
    }
}

```

4. Design Pattern

4.1 Description of Design Pattern

For Module 2, I used the Adapter Design Pattern, which is a Gang of Four Structural Pattern.

What the pattern means:

Gamma, Helm, Johnson and Vlissides (1994) describe the Adapter Pattern as one that converts the interface of a class into another interface that clients expect. Adapter lets classes work together that could not otherwise, because their interfaces do not match.

In simple words, sometimes you have two things that need to appear in the same place, but they have nothing in common. One has methods the other does not. One stores information the other has never heard of. Code that displays them ends up full of if statements asking which kind it is holding, and every new kind means another if statement in every place.

An Adapter fixes this by putting a wrapper around each thing. Every wrapper offers the exact same set of methods, called the target interface. The code doing the displaying calls those methods and never learns what is underneath.

Where this happens in my module, twice:

First, course materials. A lecturer attaches either an uploaded PDF or a link to a YouTube video. These are genuinely different. A file is stored on our disk and has a size in kilobytes and a file extension. A link is a piece of text pointing at somebody else's server, with no size and no extension. Both must appear in the same list on the course page.

Second, the calendar. A scheduled class has a start time, an end time and a room. An assignment deadline has none of those. It is a single moment, it has no duration, no location, and its date means "hand this in by" rather than "be here at". Both must appear on the same month grid.

The parts of the pattern in my code:

Pattern Role	Materials	Calendar
Target (the interface the view calls)	DisplayableMaterial	CalendarEntry
Adaptee (the mismatched thing)	CourseMaterial with a file, CourseMaterial with a URL	CourseEvent, Assignment
Adapter (the wrapper)	FileResourceAdapter, ExternalResourceAdapter	ScheduledEventAdapter, AssignmentDeadlineAdapter
Client (the code that stays simple)	courses/show.blade.php	calendar/index.blade.php

How Adapter differs from a similar pattern:

Adapter is often confused with Facade, which Module 1 uses. The difference is what each one is for. Facade takes many classes and gives you one simpler way in. Adapter takes two things that look different and makes them look the same, without making either simpler. Facade makes many things look like one. Adapter makes different things look alike.

Module 2: Adapter Pattern

Applied twice, to two mismatched pairs

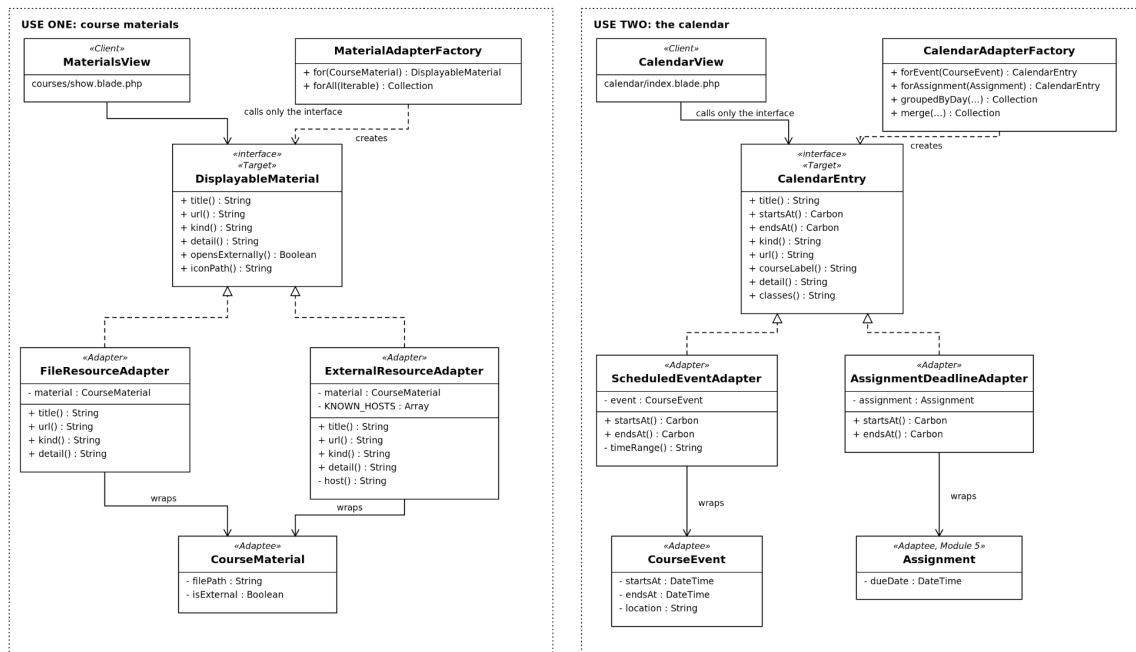


Figure 4.1: Adapter Pattern Class Diagram for Course Materials and the Calendar

4.2 Implementation of Design Pattern

1. The Target Interface for Materials (app/Patterns/Adapter/DisplayableMaterial.php)

```
namespace App\Patterns\Adapter;
```

```
/**
```

```
 * The one interface the views speak.
```

```
 *
```

```
 * A lecturer may attach an uploaded PDF or a link to something outside the
 * system, such as a YouTube video. The two have nothing in common: one is a
 * file on disk with a size and a type, the other is a URL on somebody else's
 * server. Every material is wrapped in an adapter offering this interface, so
 * the Blade view walks one list and calls the same methods on every item,
 * with no is_external check anywhere in the template.
```

```
 */
```

```
interface DisplayableMaterial
{
    /** What the student sees as the material's name. */
    public function title(): string;

    /** Where clicking it takes them. */
    public function url(): string;

    /** A short label for the kind of resource, such as PDF or YouTube. */
    public function kind(): string;

    /** Extra detail for the list: a file size, or the external host. */
    public function detail(): string;

    /** Should the link open in a new tab? True for anything off site. */
    public function opensExternally(): bool;

    /** Icon path data, so the view stays free of per type conditionals. */
    public function iconPath(): string;
}
```

2. The Adapter for an Uploaded File (app/Patterns/Adapter/FileResourceAdapter.php)

```

namespace App\Patterns\Adapter;

use App\Models\CourseMaterial;
use Illuminate\Support\Facades\Storage;

class FileResourceAdapter implements DisplayableMaterial
{
    public function __construct(private CourseMaterial $material)
    {
    }

    public function title(): string
    {
        return $this->material->title;
    }

    // A stored path turned into a public URL
    public function url(): string
    {
        return Storage::disk('public')->url($this->material->file_path);
    }

    // The file extension, so a PDF shows as PDF and a slide deck as PPTX
    public function kind(): string
    {
        $extension = strtolower(pathinfo($this->material->file_path,
        PATHINFO_EXTENSION));

        return $extension !== '' ? $extension : 'File';
    }

    // The file size, read from disk and written in human units
    public function detail(): string
    {
        if (! Storage::disk('public')->exists($this->material->file_path)) {
            return 'file missing';
        }

        $bytes = Storage::disk('public')->size($this->material->file_path);

        foreach (['B', 'KB', 'MB', 'GB'] as $unit) {
            if ($bytes < 1024) {
                return round($bytes, $unit === 'B' ? 0 : 1).' '.$unit;
            }
            $bytes /= 1024;
        }

        return round($bytes, 1).' TB';
    }

    // Our own file, so it stays in this tab

```



```
public function opensExternally(): bool
{
    return false;
}

public function iconPath(): string
{
    return 'M9 12h6m-6 4h6m2 5H7a2 2 0 01-2-2V5a2 2 0 012-2h5.586a1 1 0
01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z';
}
}
```

3. The Adapter for an External Link

(app/Patterns/Adapter/ExternalResourceAdapter.php)

```

namespace App\Patterns\Adapter;

use App\Models\CourseMaterial;

class ExternalResourceAdapter implements DisplayableMaterial
{
    // Hosts worth naming in the interface
    private const KNOWN_HOSTS = [
        'youtube.com' => 'YouTube',
        'youtu.be' => 'YouTube',
        'vimeo.com' => 'Vimeo',
        'github.com' => 'GitHub',
        'docs.google.com' => 'Google Docs',
        'drive.google.com' => 'Google Drive',
    ];

    public function __construct(private CourseMaterial $material)
    {
    }

    public function title(): string
    {
        return $this->material->title;
    }

    // The URL is stored in the same column a file path uses. The adapters
    // are what make that difference invisible to everything downstream.
    public function url(): string
    {
        return $this->material->file_path;
    }

    // There is no file extension here, so the host name is used instead.
    // This is the mismatch the Adapter is absorbing.
    public function kind(): string
    {
        $host = $this->host();

        foreach (self::KNOWN_HOSTS as $needle => $label) {
            if (str_contains($host, $needle)) {
                return $label;
            }
        }

        return 'Link';
    }

    // No size exists for a link, so the host is shown in its place
    public function detail(): string

```

```
{
    return $this->host() !== " ? $this->host() : 'external resource';
}

// Somebody else's site, so it opens in a new tab
public function opensExternally(): bool
{
    return true;
}

public function iconPath(): string
{
    return 'M10 6H6a2 2 0 00-2 2v10a2 2 0 002 2h10a2 2 0 002-2v-4M14 4h6m0
0v6m0-6L10 14';
}

private function host(): string
{
    return str_replace('www.', "", (string) parse_url($this->material->file_path,
PHP_URL_HOST));
}
}
```

4. Choosing the Right Adapter (app/Patterns/Adapter/MaterialAdapterFactory.php)

```

namespace App\Patterns\Adapter;

use App\Models\CourseMaterial;
use Illuminate\Support\Collection;

class MaterialAdapterFactory
{
    // This is the ONLY place in the whole system that looks at is_external.
    // Everything after this point deals only in DisplayableMaterial, which
    // is exactly what the Adapter pattern is buying.
    public static function for(CourseMaterial $material): DisplayableMaterial
    {
        return $material->is_external
            ? new ExternalResourceAdapter($material)
            : new FileResourceAdapter($material);
    }

    public static function forAll(iterable $materials): Collection
    {
        return collect($materials)->map(fn (CourseMaterial $material) => [
            'material' => $material,
            'display' => self::for($material),
        ]);
    }
}

```

5. The View, Which Never Asks Which Kind It Has

(resources/views/courses/show.blade.php)

```

@foreach ($category['items'] as $item)
    @php $display = $item['display']; @endphp

    <a href="{{ $display->url() }}"
        @if ($display->opensExternally()) target="_blank" rel="noopener noreferrer" @endif>

        <svg viewBox="0 0 24 24">
            <path d="{{ $display->iconPath() }}" />
        </svg>

        <span>{{ $display->title() }}</span>
        <span>{{ $display->kind() }} &middot; {{ $display->detail() }}</span>
    </a>
@endforeach

```

6. The Second Use of the Pattern

(app/Patterns/Adapter/AssignmentDeadlineAdapter.php)

```

namespace App\Patterns\Adapter;

use App\Models\Assignment;
use Carbon\CarbonInterface;

/**
 * ADAPTEE: Assignment, which is not a diary entry at all. It is a piece of
 * work with a deadline, and this adapter presents it as though it were one.
 *
 * The mismatch being absorbed: an assignment has no end time, no room and no
 * link, and its date means "by" rather than "at".
 */
class AssignmentDeadlineAdapter implements CalendarEntry
{
    public function __construct(private Assignment $assignment)
    {
    }

    public function title(): string
    {
        return $this->assignment->title;
    }

    public function startsAt(): CarbonInterface
    {
        return $this->assignment->due_date;
    }

    // A deadline is a moment, not a span. Returning a fake end time would
    // draw it as a block on the grid and imply a duration nobody stated.
    public function endsAt(): ?CarbonInterface
    {
        return null;
    }

    public function kind(): string
    {
        return 'Assignment due';
    }

    public function url(): ?string
    {
        return route('assignments.show', $this->assignment->id);
    }

    public function courseLabel(): ?string
    {
        return $this->assignment->course?->code;
    }
}

```

```
public function detail(): string
{
    return 'Due '.$this->assignment->due_date->format('g:ia');
}

// Amber, so a deadline never reads as just another class on the grid
public function classes(): string
{
    return 'border-amber-300 bg-amber-50 text-amber-900';
}
}
```

4.3 Justification of Design Pattern

1. The two things really are different, and the difference is not going away. A stored file has a size in bytes and a file extension. An external link has a host name and neither of the other two. There is no sensible way to make one into the other, and no sensible way to store both in a single shape without losing information. The Adapter is the pattern for exactly this situation: two mismatched sources that must be presented as one.
2. The view stays clean, which is the visible payoff. Without adapters, the materials template would need an if statement asking whether each item is external, then a second one to decide the icon, a third for whether to open a new tab, and a fourth to choose between showing a file size or a host name. That is four branches in a template, repeated in the calendar template as well. With adapters there are none. `courses/show.blade.php` calls `title()`, `url()`, `kind()` and `detail()` and never learns what it is holding.
3. Only one place in the whole system inspects `is_external`. That place is `MaterialAdapterFactory::for()`, a single line. Everything downstream deals in `DisplayableMaterial`. If a third kind of material were added later, such as an embedded video hosted by the college, it would mean writing one new adapter class and adding one branch to the factory. No template, no controller and no other adapter would change. That follows the Open Closed Principle.
4. The calendar case proves the pattern was worth having, because it prevented a real bug. The obvious way to put deadlines on a calendar is to copy each one into the events table. That works exactly until a lecturer moves a due date, at which point the calendar shows the old date and the assignment page shows the new one, and nobody can tell which is right. Adapting Module 5's Assignment instead means there is only ever one date. Change it and the calendar moves with it, because there is nothing to keep in step.
5. It respects the module boundary the specification draws. `AssignmentDeadlineAdapter` reads Module 5's Assignment without writing to it and without copying it. Module 5 stays the sole owner of assignments, and Module 2 still gets to display them. The Adapter is what makes that possible without either module reaching into the other.

5. Software Security

5.1 Potential Threat/Attack

Threat 1: Malicious File Upload Leading to Remote Code Execution (OWASP A03: Injection)

Attack Description:

Module 2 lets a lecturer upload course material, and that upload is the most dangerous input in the whole module. The reason is where the file ends up. Uploaded materials are written to the public disk, and public/storage is a symbolic link into the web root so that students can open a PDF by clicking it. Anything written there is reachable at a URL.

Under Apache with PHP, which is how this project runs on XAMPP, a file reachable at a URL and ending in .php is not downloaded. It is executed by the server. So an attacker who can upload a file called shell.php containing `<?php system($_GET['cmd']); ?>` can then visit that file's URL and run any command they like on the machine, read the database configuration, and take the entire system.

The attacker does not even need to be a lecturer. Any of the login attacks that Module 1 defends against would hand them a lecturer account, and this upload turns a stolen account into full control of the server.

A related and easier attack uses HTML rather than PHP. A file called notes.html containing a script would be served from this site's own address, so any script inside it runs with the site's own origin and can read the session cookie of every student who opens it.

Risk Impact:

Complete compromise of the server. An attacker gains the ability to read and change any record in the database, including grades and certificates, and to read the .env file containing the application key and database password.

Threat 2: Broken Access Control Through Guessed Record IDs (OWASP A01: Broken Access Control)**Attack Description:**

Almost every page in Module 2 is reached by a URL containing a record's ID number, such as `/courses/4`, `/calendar/events/12` or `/announcements`. Those IDs are sequential and completely predictable.

This creates what is called an Insecure Direct Object Reference. A student enrolled in course 4 sees `/courses/4` in the address bar and simply types `/courses/5` instead. If the application fetches course 5 and displays it without asking whether this particular student is allowed to see it, then a student can read every course in the college, including materials for subjects they never enrolled in and announcements meant for other cohorts.

The same trick applies to the calendar. Guessing `/calendar/events/12` would reveal a meeting belonging to a course the student is not part of, including its description and its meeting link, which would let them join a class they are not enrolled in.

It also applies in the other direction, between staff. Lecturer A viewing `/courses/4/edit` could change the URL to a course belonging to Lecturer B and edit somebody else's course, or upload material into it, unless ownership is checked on every write.

The reason this is such a common vulnerability is that it needs no tools at all. It is done by editing the address bar, so anyone who notices the pattern can try it.

Risk Impact:

Unauthorised disclosure of course materials, announcements and meeting links across the whole institution, and unauthorised modification of courses belonging to other lecturers.

5.2 Secure Coding Practice

Secure Practice 1: An Allow-List of Upload Types Checked Against the Real File Content

OWASP Category: File Management. The defence is a strict list of what may be uploaded, rather than a list of what may not.

```
// app/Http/Controllers/CourseMaterialController.php

/**
 * File types a lecturer may upload as course material.
 *
 * An allow-list, never a block-list. A block-list has to anticipate every
 * dangerous extension, and it only takes one that was forgotten (.phtml,
 * .phar, .htaccess) for the defence to fail. An allow-list fails the other
 * way: something new is refused until it is deliberately added.
 */
private const ALLOWED_UPLOAD_TYPES = [
    'pdf',
    'doc', 'docx',
    'ppt', 'pptx',
    'xls', 'xlsx', 'csv',
    'txt', 'md',
    'png', 'jpg', 'jpeg', 'gif', 'webp',
    'zip',
];

$data = $request->validate([
    // ...
    // SECURITY (Module 2): the allow-list is the whole defence here.
    // mimes: checks the extension against the file's real MIME type, so
    // renaming shell.php to shell.pdf does not get past it either.
    'file' => [
        'required_if:source,file',
        'file',
        'max:20480',
        'mimes':implode(',', self::ALLOWED_UPLOAD_TYPES),
    ],
    // SECURITY (Module 2): only http(s). A javascript: or data: URL saved
    // here would become a scripted link rendered as an ordinary course
    // material for every student on the course.
    'url' => [
        'required_if:source,link',
        'nullable',
        'url',
        'max:255',
        'starts_with:http://,https://',
    ],
]);
```

Why an allow-list rather than a block-list. A block-list has to think of every dangerous extension in advance. Miss one, such as .phtml or .phar, and the defence is gone. An

allow-list fails safely instead: anything not on the list is refused, so a file type nobody thought about is rejected by default rather than accepted by default.

Why mimes: rather than checking the file name. Laravel's mimes: rule reads the file's actual content type and compares it to the extension. An attacker who renames shell.php to shell.pdf is still refused, because the content is still PHP. Checking the name alone would be trivially bypassed.

Why the URL rule matters too. The same form accepts an external link, stored in the same database column. Without starts_with:http://,https://, a lecturer or an attacker with a lecturer account could save javascript:alert(document.cookie) as a material. The Adapter would then render it as an ordinary clickable resource, and every student who clicked it would run the attacker's script.

Where the rest of the module already stood. Student assignment submissions were never exposed to this, because they are written to the local disk, which is not inside the web root, and are served by SubmissionController::download() after it checks that the requester is either the student who submitted or the lecturer marking it. Materials needed fixing because they are deliberately public to the class.

Secure Practice 2: Checking Enrolment and Ownership on Every Single Request

OWASP Category: Access Control. Every route that reaches a course record checks the requester against that record before returning anything. There is no page that trusts an ID from the URL.

```
/ app/Http/Controllers/CourseController.php

/**
 * SECURITY (Module 2): who may look at this course.
 *
 * Checked on every request rather than once at login, because the ID comes
 * from the URL and the URL is under the visitor's control.
 */
private function authoriseAccess(Request $request, Course $course): void
{
    $user = $request->user();

    $allowed = $course->instructor_id === $user->id // the lecturer who owns it
        || $user->can('analytics.view_system') // an administrator
        || ($user->can('material.view') && $course->hasStudent($user)); // an enrolled
    student

    abort_unless($allowed, 403, 'You are not enrolled in this course.');
```

```
}

/**
 * SECURITY (Module 2): who may CHANGE this course.
 *
 * Two checks, not one. The permission key says this role may edit courses
 * in general. The ownership test says this lecturer may edit THIS course.
 * A lecturer holding course.update is still refused on somebody else's.
 */
private function authoriseOwner(Request $request, Course $course, string $permission):
void
{
    abort_unless($request->user()->can($permission), 403);
    abort_unless($course->instructor_id === $request->user()->id, 403,
        'This course belongs to another instructor.');
```

```
}
```

The two checks answer different questions, and both are needed. The permission key asks whether this role is allowed to do this kind of thing at all, and it is stored in the database so an administrator can change it. The ownership test asks whether this particular person is allowed to do it to this particular record. A system with only the first would let any lecturer edit any course. A system with only the second would ignore the permission matrix entirely.

The calendar applies the same idea through a query scope, so that guessing an event ID reveals nothing the month grid would not already have shown:

```
// app/Models/CourseEvent.php

/**
 * SECURITY (Module 2): events this user is entitled to see. Global ones,
 * their own courses' ones, and for an administrator, everything.
 */
public function scopeVisibleTo(Builder $query, User $user): Builder
{
    return $query->where(function (Builder $q) use ($user) {
        $q->whereNull('course_id');           // institution-wide

        if ($user->can('analytics.view_system')) {
            $q->orWhereNotNull('course_id');    // administrator
            return;
        }

        if ($user->can('course.enroll')) {
            $q->orWhereIn('course_id', $user->courses()->pluck('courses.id'));
        }

        if ($user->can('course.create')) {
            $q->orWhereIn('course_id', $user->coursesTeaching()->pluck('id'));
        }
    });
}
```

```
// app/Http/Controllers/CalendarController.php

// SECURITY (Module 2): the detail page asks the SAME question the grid asks,
// rather than a second, looser rule that could drift away from it.
abort_unless(
    CourseEvent::visibleTo($request->user())->whereKey($event->getKey())->exists(),
    403,
    'That event belongs to a course you are not part of.'
);
```

Reusing the scope matters more than it looks. If the detail page had its own separate access rule, the two could disagree the moment either was edited, and an event hidden from the grid could still be reachable by its own URL. Asking the same question in both places means they can never fall out of step.

6. Web Services

Module 2 works with web services in both directions, and both are built and working in the code.

As a provider, it exposes `getCourseInfo`. Module 2 is the only owner of course data, so when Module 1 needs a course title to print on a certificate, or Module 4 needs one to label a quiz, they ask this service rather than reading Module 2's tables.

As a consumer, it calls Module 5's `getCourseAnalytics`. Module 2 owns course content and knows nothing about how a mark is worked out. Rather than copying Module 5's grading rules, which would then have to be kept in step every time the grade scale changed, it asks Module 5 for the figures and shows whatever comes back.

I chose REST with JSON over SOAP because it is lighter, needs no WSDL contract file or XML wrapper, and can be called straight from a browser, from Postman, or from PHP.

Both halves follow one shared Interface Agreement held in `app/Support/ifa.php`. Every request carries a `requestID` and a `timeStamp`. Every response carries a status of S, F or E, a `timeStamp`, and the `requestID` echoed back so a caller can match an answer to the question it asked.

6.1 Web Service Exposure

Interface Agreement (IFA) for Service Exposure

	Description
Protocol	RESTful Web Service (JSON over HTTP)
Function Description	<i>Returns the course code, title, student count and optionally the lecturer, for a given course ID</i>
Source Module	Module 2: Academic Resources Repository Module
Target Module	Module 1 (Identity and Credentialing), Module 4 (Skill Assessment and Quiz)
HTTP Method & URL	GET <code>/api/courses/info</code>
Controller Action	<code>App\Http\Controllers\Api\CourseApiController@info</code>
Function Name	<code>getCourseInfo</code>
Authentication	Shared key in an X-API-Key header

Web Services Request Parameters (IFA Requirement)

Field Name	Field Type	Mandatory/ Optional	Description	Format
courseId	Integer	Mandatory	The ID of the course to look up.	Whole number above 0
queryFlag	Integer	Mandatory	How much detail is needed.	1: code, title and student count 2: also the lecturer
requestID	String	Mandatory	A unique ID so the request can be traced.	Letters, numbers, hyphens. Max 64
timeStamp	String	Mandatory	The time the request was made.	YYYY-MM-DDTHH:MM:SSZ

Web Services Response Parameters (IFA Requirement)

Field Name	Field Type	Mandatory / Optional	Description	Format / Values
status	String	Mandatory	Whether the request worked.	S for Success, F for Fail, E for Error
timeStamp	String	Mandatory	The time the answer was created.	YYYY-MM-DDTHH:MM:SSZ
data.requestID	String	Mandatory	The request ID sent back, for matching.	Letters, numbers, hyphens
data.courseCode	String	Mandatory	The public course code.	Letters and numbers, e.g. BMIT3173
data.courseTitle	String	Mandatory	The full course name.	Letters and numbers
data.studentCount	Integer	Mandatory	How many students are enrolled.	0 or above
data.instructorName	String	Optional	The lecturer's name.	Only sent when queryFlag is 2
data.instructorEmail	String	Optional	The lecturer's published address.	Only sent when queryFlag is 2

Consumption Code Implementation (app/Support/CourseInfoClient.php)

```

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Course;
use App\Support\Ifa;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;
use Throwable;

class CourseApiController extends Controller
{
    public function info(Request $request): JsonResponse
    {
        // Ifa::baseRules() supplies the two fields every service demands,
        // so all five members validate them the same way.
        $validator = validator($request->all(), Ifa::baseRules() + [
            'courseId' => ['required', 'integer', 'min:1'],
            'queryFlag' => ['required', 'integer', 'in:1,2'],
        ]);

        if ($validator->fails()) {
            return Ifa::fail($request, ['errors' => $validator->errors()->all()]);
        }

        try {
            $course = Course::with('instructor')->find($request->integer('courseId'));

            if ($course === null) {
                return Ifa::fail($request, [
                    'message' => 'No course exists with that ID.',
                ], 404);
            }

            $data = [
                'courseCode' => $course->code,
                'courseTitle' => $course->title,
                'studentCount' => $course->students()->count(),
            ];

            // SECURITY (Module 2): the class code is never returned at any
            // flag. Holding it grants enrolment, so it must not travel over
            // an integration channel.
            if ($request->integer('queryFlag') === 2) {
                $data['instructorName'] = $course->instructor?->name;
                $data['instructorEmail'] = $course->instructor?->contactEmail();
            }

            return Ifa::success($request, $data);

        } catch (Throwable $e) {
            Log::error('getCourseInfo failed', ['error' => $e->getMessage()]);
        }
    }
}

```



```

        // The exception message goes to the log, never to the caller.
        // A stack trace in an API response is an information leak.
        return lfa::error($request);
    }
}
}

```

```

// routes/api.php
Route::middleware('api.key')->group(function () {
    Route::get('/courses/info', [CourseApiController::class, 'info'])
        ->name('api.courses.info');
    // ... the other internal services
});

```

Real response from the running system:

```

{
  "status": "S",
  "timeStamp": "2026-09-05T16:30:17Z",
  "data": {
    "requestID": "CRS-REQ-11223",
    "courseCode": "BMIT3173",
    "courseTitle": "Integrative Programming",
    "studentCount": 33,
    "instructorName": "Malarvili A/P Nallayan",
    "instructorEmail": "malarvili@tarc.edu.my"
  }
}

```

A call with no API key is refused:

```

{
  "status": "F",
  "timeStamp": "2026-09-05T16:30:36Z",
  "data": {
    "requestID": "X",
    "message": "A valid X-API-Key header is required for this service."
  }
}

```

6.2 Web Service Consumption

Interface Agreement (IFA) for Service Consumption

	Description
Protocol	RESTful Web Service (JSON over HTTP)
Function Description	Returns cohort marks for one course: average, highest, lowest, pass count and grade distribution
Source Module	Module 5: Academic Progress Analytics Module
Consuming Module	Module 2: Academic Resources Repository Module
HTTP Method & URL	GET /api/analytics/course
Client Class	App\Support\Api\CourseAnalyticsClient@fetch
Function Name	getCourseAnalytics
Protocol	RESTful Web Service (JSON over HTTP)

Web Services Request Parameters (IFA Requirement)

Field Name	Field Type	Mandatory/Optional	Description	Format
courseId	Integer	Mandatory	The course to report on.	Whole number above 0
requestID	String	Mandatory	A tracking ID made by Module 2, prefixed ANL-REQ.	Letters, numbers, hyphens
timeStamp	String	Mandatory	The time the request was sent.	YYYY-MM-DDTHH:MM:SSZ

Web Services Response Parameters (IFA Requirement for Consumption)

Field Name	Field Type	Mandatory / Optional	Description	Format / Values
status	String	Mandatory	Whether the request worked.	S for Success, F for Fail, E for Error
timeStamp	String	Mandatory	The time the answer was created.	YYYY-MM-DDTHH:MM:SSZ
data.courseCode	String	Mandatory	The course reported on.	Letters and numbers
data.gradedCount	Integer	Mandatory	How many marks exist for the course.	0 or above
data.averageScore	Double	Optional	The class average.	0.00 to 100.00
data.averageGrade	String	Optional	The letter for that average.	A, A-, B+, B and so on
data.highestScore	Double	Optional	The best mark in the class.	0.00 to 100.00
data.lowestScore	Double	Optional	The lowest mark in the class.	0.00 to 100.00
data.passCount	Integer	Optional	How many marks were a pass.	0 or above
data.distribution	Object	Optional	Counts per letter family.	{"A":17,"B":10,"C":10,"D":2,"F":2}

Consumption Code Implementation (app/Support/CourseInfoClient.php)

```

namespace App\Support\Api;

/**
 * CONSUMES Module 5's getCourseAnalytics service.
 *
 * Module 2 owns course content and knows nothing about how a mark is worked
 * out. Rather than duplicating Module 5's grading logic, which would then
 * have to be kept in step every time the grade scale changed, it asks Module
 * 5 for the figures and displays whatever comes back.
 */
class CourseAnalyticsClient extends ServiceClient
{
    // Stamped on this client's request IDs, so a call in Module 5's log can
    // be traced back to Module 2 as the caller.
    protected function requestPrefix(): string
    {
        return 'ANL-REQ';
    }

    public function fetch(int $courseId): ?array
    {
        return $this->get('/analytics/course', [
            'courseId' => $courseId,
        ]);
    }
}

```

```

// app/Support/Api/ServiceClient.php (extract)

private function send(string $method, string $path, array $parameters): ?array
{
    $url = rtrim((string) config('services.internal_api.base_url'), '/') . '/' . ltrim($path, '/');

    // The two fields the IFA makes mandatory on every request
    $payload = $parameters + Ifa::requestEnvelope($this->requestPrefix());

    try {
        $response = $this->request()->{$method}($url, $payload);
    } catch (Throwable $e) {
        // A refused connection or a timeout lands here
        Log::warning('Web service call failed', [
            'url' => $url, 'requestID' => $payload['requestID'],
            'error' => $e->getMessage(),
        ]);

        return null;
    }

    $body = $response->json();

    // The IFA status is the contract, not the HTTP code. A provider can

```

```

// return 200 with a status of F, meaning it understood the question
// and could not answer it.
if (! lfa::succeeded($body)) {
    return null;
}

return $body['data'] ?? null;
}

private function request(): PendingRequest
{
    return Http::acceptJson()
        ->timeout((int) config('services.internal_api.timeout', 10))
        ->withHeaders(['X-API-Key' => (string) config('services.internal_api.key')]);
}

```

Every client returns null when a call does not succeed, and never throws. This is deliberate. A module must keep working when another member's service is down, so a lecturer can still open a course page while the analytics service is restarting. The page simply shows no performance panel instead of showing an error.

Real response from the running system:

```

{
    "status": "S",
    "timeStamp": "2026-09-05T16:30:35Z",
    "data": {
        "requestID": "ANL-REQ-99001",
        "courseCode": "BMIT3173",
        "gradedCount": 41,
        "averageScore": 65.9,
        "averageGrade": "B",
        "highestScore": 83.33,
        "lowestScore": 24,
        "passCount": 39,
        "distribution": { "A": 17, "B": 10, "C": 10, "D": 2, "F": 2 }
    }
}

```

Figure 6.3: Postman Showing Module 2 Consuming Module 5's Analytics Service

7. References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

Laravel LLC. (2026). Laravel 12.x documentation: Eloquent ORM, validation and file storage. <https://laravel.com/docs/12.x>

OWASP Foundation. (2021). OWASP Top 10:2021 The ten most critical web application security risks. Open Web Application Security Project. <https://owasp.org/Top10/>

OWASP Foundation. (2022). OWASP secure coding practices quick reference guide (Version 2.1). <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

United Nations. (2015). Transforming our world: The 2030 agenda for sustainable development (Goal 4: Quality Education). United Nations Department of Economic and Social Affairs. <https://sdgs.un.org/goals/goal4>

8. Appendices

Appendix A: Automated Testing Results

```
c:\xampp\htdocs\edusystem>php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true 0.01s

PASS Tests\Feature\Auth\AuthenticationTest
✓ login screen can be rendered 0.59s
✓ users can authenticate using the login screen 0.08s
✓ users can not authenticate with invalid password 0.24s
✓ users can logout 0.03s

PASS Tests\Feature\Auth\EmailVerificationTest
✓ email verification screen can be rendered 0.04s
✓ email can be verified 0.04s
✓ email is not verified with invalid hash 0.03s

PASS Tests\Feature\Auth\PasswordConfirmationTest
✓ confirm password screen can be rendered 0.04s
✓ password can be confirmed 0.03s
✓ password is not confirmed with invalid password 0.23s

PASS Tests\Feature\Auth\PasswordResetTest
✓ reset password link screen can be rendered 0.03s
✓ reset password link can be requested 0.25s
✓ reset password screen can be rendered 0.26s
✓ password can be reset with valid token 0.28s

PASS Tests\Feature\Auth\PasswordUpdateTest
✓ password can be updated 0.04s
✓ correct password must be provided to update password 0.03s

PASS Tests\Feature\Auth\RegistrationTest
✓ open registration route does not exist 0.04s
✓ an unknown token is rejected 0.03s
✓ an expired invitation is refused 1.66s
✓ an already accepted invitation is refused 0.56s
✓ a valid invitation creates the account with its fixed role 0.05s
✓ a token cannot be redeemed twice 0.60s

PASS Tests\Feature\AwardAndActivityNotificationTest
✓ earning a certificate notifies the holder 0.53s
✓ earning a badge notifies the student 0.05s
✓ marking submitted work notifies the student 0.04s
✓ a marked quiz does not notify because the result is already on screen 0.03s
```

```

✓ inviting a student to a course notifies them 0.03s
✓ a switched off preference still suppresses the new types 0.03s

PASS Tests\Feature\AwardRuleTest
✓ an admin defined average score rule awards a badge 0.06s
✓ an admin defined quizzes completed rule counts distinct quizzes 0.05s
✓ an admin defined certificate rule mints a real credential 0.41s
✓ a certificate rule does not mint twice 0.56s
✓ a certificate rule is never handed out as a badge 0.50s
✓ deactivating a rule stops it without removing awards already made 0.06s

PASS Tests\Feature\CalendarEventDetailTest
✓ an enrolled student can open an events detail page 0.07s
✓ a student cannot open an event for a course they are not in 0.04s
✓ a lecturer cannot open an event for another lecturers course 0.04s
✓ an institution wide event is visible to everyone 0.05s
✓ a meeting shows a join button 0.05s
✓ an event with no link shows no join button 0.06s
✓ a malformed meeting link does not crash or render a button 0.06s
✓ a javascript url is never rendered as a join button 0.05s
✓ a student does not see the names of their classmates 0.06s

PASS Tests\Feature\CourseEnrolmentTest
✓ a student cannot leave a course themselves 0.05s
✓ the owning lecturer can remove a student 0.04s
✓ a lecturer cannot remove a student from another lecturers course 0.05s
✓ a student cannot remove a classmate 0.04s
✓ removing somebody who is not enrolled is a 404 0.04s

PASS Tests\Feature\CourseMaterialUploadTest
✓ a lecturer can upload a normal document 0.06s
✓ a php file is refused 0.04s
✓ a php file renamed to look like a pdf is still refused 0.05s
✓ an html file is refused 0.04s
✓ a javascript url cannot be saved as an external material 0.04s
✓ an ordinary external link is accepted 0.04s
✓ a lecturer cannot upload to another lecturers course 0.04s

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 0.04s

PASS Tests\Feature\ProfileTest
✓ profile page is displayed 0.06s
✓ profile information can be updated 0.04s
✓ email verification status is unchanged when the email address is unchanged 0.04s
✓ user can delete their account 0.03s
✓ correct password must be provided to delete account 0.03s

PASS Tests\Feature\ProfileTest
✓ profile page is displayed 0.06s
✓ profile information can be updated 0.04s
✓ email verification status is unchanged when the email address is unchanged 0.04s
✓ user can delete their account 0.03s
✓ correct password must be provided to delete account 0.03s

PASS Tests\Feature\SubjectExpertBadgeTest
✓ passing every quiz in the subject awards the badge 0.06s
✓ attempting without passing does not award it 0.04s
✓ a subject with no quizzes awards nothing 0.03s
✓ resubmitting does not award a second copy 0.05s
✓ a quiz added afterwards does not revoke the badge 0.05s
✓ the badge is scoped to its own subject 0.06s

PASS Tests\Feature\WebServiceTest
✓ every response carries status timestamp and the request id 0.05s
✓ a request without the mandatory ifa fields is refused 0.03s
✓ an internal service refuses a caller with no api key 0.03s
✓ an internal service refuses a wrong api key 0.03s
✓ the credential service is public and needs no key 0.43s
✓ detail flag one does not disclose the holder 0.45s
✓ detail flag two returns the holder 0.54s
✓ an unknown credential reports not found rather than erroring 0.03s
✓ a tampered credential is reported over the service 0.45s
✓ the quiz service returns the best attempt 0.06s
✓ the analytics service returns cohort figures only 0.05s
✓ the notification service writes to the inbox 0.03s
✓ the notification service refuses a type outside the allow list 0.03s
✓ module 1 consumes module 2s course info service 0.08s
✓ module 5 consumes module 1s credential service 0.42s
✓ module 3 consumes module 4s quiz result service 0.05s
✓ a client returns null rather than throwing when the service is unreachable 1.07s

Tests: 86 passed (200 assertions)
Duration: 13.37s

```

Figure 8.1: Terminal Output of php artisan test Showing 86 Passing Tests


```

c:\xampp\htdocs\edusystem>php artisan test --filter=CourseMaterialUploadTest

PASS Tests\Feature\CourseMaterialUploadTest
✓ a lecturer can upload a normal document 0.61s
✓ a php file is refused 0.04s
✓ a php file renamed to look like a pdf is still refused 0.04s
✓ an html file is refused 0.04s
✓ a javascript url cannot be saved as an external material 0.04s
✓ an ordinary external link is accepted 0.04s
✓ a lecturer cannot upload to another lecturers course 0.05s

Tests: 7 passed (18 assertions)
Duration: 1.05s

c:\xampp\htdocs\edusystem>php artisan test --filter=CourseEnrolmentTest

PASS Tests\Feature\CourseEnrolmentTest
✓ a student cannot leave a course themselves 0.58s
✓ the owning lecturer can remove a student 0.04s
✓ a lecturer cannot remove a student from another lecturers course 0.04s
✓ a student cannot remove a classmate 0.04s
✓ removing somebody who is not enrolled is a 404 0.04s

Tests: 5 passed (9 assertions)
Duration: 0.94s

c:\xampp\htdocs\edusystem>php artisan test --filter=CalendarEventDetailTest

PASS Tests\Feature\CalendarEventDetailTest
✓ an enrolled student can open an events detail page 0.57s
✓ a student cannot open an event for a course they are not in 0.04s
✓ a lecturer cannot open an event for another lecturers course 0.04s
✓ an institution wide event is visible to everyone 0.04s
✓ a meeting shows a join button 0.05s
✓ an event with no link shows no join button 0.04s
✓ a malformed meeting link does not crash or render a button 0.04s
✓ a javascript url is never rendered as a join button 0.04s
✓ a student does not see the names of their classmates 0.05s

Tests: 9 passed (22 assertions)
Duration: 1.15s

```

Figure 8.2: Terminal Output of own modules php artisan test

Appendix B: GitHub Repository URL

Team Repository: <https://github.com/NickFoo0924/edusystem>

Branch: master